```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from google.colab import files

uploaded = files.upload()

filename = next(iter(uploaded))

df = pd.read_csv("Online_Retail.csv")

print("Dataset imported successfully!")
print("File name:", filename)

display(df.head())
```

Choose Files Online_Retail.csv

Online_Retail.csv(text/csv) - 47217062 bytes, last modified: 10/29/2026 - 100% done

```
Saving Online_Retail.csv to Online_Retail.csv
Dataset imported successfully!
File name: Online_Retail.csv
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/01/2010 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/01/2010 08:26:00	2.75	17850.0	United Kingdom

```
df_original = df.copy()

print("Original dataset backup created!")

print("Dataset shape:", df_original.shape)
display(df_original.head())
```

```
Original dataset backup created!
Dataset shape: (541909, 8)
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/01/2010 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/01/2010 08:26:00	2.75	17850.0	United Kingdom

```
# First five rows
print("First five rows:")
display(df_original.head())

# First three rows
print("\nFirst three rows:")
display(df_original.head(3))

# Last five rows
print("\nLast five rows:")
display(df_original.tail())

# Dataset dimensions
print("\nDataset shape:", df_original.shape)

# Column names
print("\nColumn names:")
print(df_original.columns.tolist())

# Dataset information
print("\nDataset information:")
```

```
df_original.info()

# Descriptive statistics
print("\nDescriptive statistics:")
display(df_original.describe(include="all"))
```

First five rows:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/01/2010 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/01/2010 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom

First three rows:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/01/2010 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/01/2010 08:26:00	2.75	17850.0	United Kingdom

Last five rows:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	12/09/2011 12:50:00	0.85	12680.0	France
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	12/09/2011 12:50:00	2.10	12680.0	France
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	12/09/2011 12:50:00	4.15	12680.0	France
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	12/09/2011 12:50:00	4.15	12680.0	France
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	12/09/2011 12:50:00	4.95	12680.0	France

```
Dataset shape: (541909, 8)

Column names:
['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate', 'UnitPrice', 'CustomerID', 'Country']

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    541909 non-null  object
 1   StockCode    541909 non-null  object
 2   Description  540455 non-null  object
 3   Quantity     541909 non-null  int64
 4   InvoiceDate  541909 non-null  object
 5   UnitPrice    541909 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      541909 non-null  object
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
```

Descriptive statistics:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
count	541909	541909	540455	541909.000000	541909	541909.000000	406829.000000	541909
unique	25900	4070	4223	NaN	23260	NaN	NaN	38
top	573585	85123A	WHITE HANGING HEART T-LIGHT HOLDER	NaN	10/31/2011 14:41:00	NaN	NaN	United Kingdom
freq	1114	2313	2369	NaN	1114	NaN	NaN	495478
mean	NaN	NaN	NaN	9.552250	NaN	4.611114	15287.690570	NaN
std	NaN	NaN	NaN	218.081158	NaN	96.759853	1713.600303	NaN

```
print("Missing values:")
print(df_original.isnull().sum())
```

```
print("\nTotal missing values:")
print(df_original.isnull().sum().sum())

print("\nNumber of duplicate records:")
print(df_original.duplicated().sum())
```

```
Missing values:
InvoiceNo           0
StockCode           0
Description      1454
Quantity            0
InvoiceDate         0
UnitPrice           0
CustomerID     135080
Country             0
dtype: int64

Total missing values:
136534

Number of duplicate records:
5268
```

```
df_clean = df_original.copy()

duplicates_before = df_clean.duplicated().sum()

df_clean = df_clean.drop_duplicates().copy()

print("Duplicates removed:", duplicates_before)
print("Shape after removing duplicates:", df_clean.shape)

text_columns = df_clean.select_dtypes(
    include=["object", "string"]
).columns

for col in text_columns:
    df_clean[col] = df_clean[col].astype("string").str.strip()

if "Country" in df_clean.columns:
    df_clean["Country"] = df_clean["Country"].str.title()

if "Description" in df_clean.columns:
    df_clean["Description"] = df_clean["Description"].str.title()

print("\nText cleaning completed!")
display(df_clean.head())
```

```
Duplicates removed: 5268
Shape after removing duplicates: (536641, 8)

Text cleaning completed!
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	White Hanging Heart T-Light Holder	6	12/01/2010 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	White Metal Lantern	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	Cream Cupid Hearts Coat Hanger	8	12/01/2010 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	Knitted Union Flag Hot Water Bottle	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	Red Woolly Hottie White Heart.	6	12/01/2010 08:26:00	3.39	17850.0	United Kingdom

```
if "InvoiceDate" in df_clean.columns:

    original_dates = df_clean["InvoiceDate"].copy()

    df_clean["InvoiceDate"] = pd.to_datetime(
        df_clean["InvoiceDate"],
        errors="coerce"
    )

    invalid_dates = (
        original_dates.notna() &
        df_clean["InvoiceDate"].isna()
    ).sum()
```

```
    print("Date conversion completed!")
    print("InvoiceDate data type:", df_clean["InvoiceDate"].dtype)
    print("Invalid dates:", invalid_dates)

    display(df_clean[["InvoiceDate"]].head())

else:
    print("InvoiceDate column not found!")
```

```
Date conversion completed!
InvoiceDate data type: datetime64[ns]
Invalid dates: 0
```

	InvoiceDate
0	2010-12-01 08:26:00
1	2010-12-01 08:26:00
2	2010-12-01 08:26:00
3	2010-12-01 08:26:00
4	2010-12-01 08:26:00

```
numeric_columns = [
    "Quantity",
    "UnitPrice",
    "CustomerID"
]

for col in numeric_columns:

    if col in df_clean.columns:

        original_values = df_clean[col].copy()

        df_clean[col] = pd.to_numeric(
            df_clean[col],
            errors="coerce"
        )

        invalid_count = (
            original_values.notna() &
            original_values.astype("string").str.strip().ne("") &
            df_clean[col].isna()
        ).sum()

        print("\nColumn:", col)
        print("Data type:", df_clean[col].dtype)
        print("Invalid values converted to NaN:", invalid_count)

print("\nNumeric conversion completed!")

display(df_clean.head())
```

```
Column: Quantity
Data type: int64
Invalid values converted to NaN: 0

Column: UnitPrice
Data type: float64
Invalid values converted to NaN: 0

Column: CustomerID
Data type: float64
Invalid values converted to NaN: 0

Numeric conversion completed!
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	White Hanging Heart T-Light Holder	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	White Metal Lantern	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	Cream Cupid Hearts Coat Hanger	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	Knitted Union Flag Hot Water Bottle	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	Red Woolly Hottie White Heart.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

```
price = df_clean["UnitPrice"].dropna()

Q1 = price.quantile(0.25)
Q3 = price.quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - (1.5 * IQR)
upper_bound = Q3 + (1.5 * IQR)

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

outliers = df_clean[
    (df_clean["UnitPrice"] < lower_bound) |
    (df_clean["UnitPrice"] > upper_bound)
]

print("\nNumber of potential outliers:", len(outliers))

display(outliers.head(10))
```

```
Q1: 1.25
Q3: 4.13
IQR: 2.88
Lower Bound: -3.0700000000000003
Upper Bound: 8.45

Number of potential outliers: 39450
```

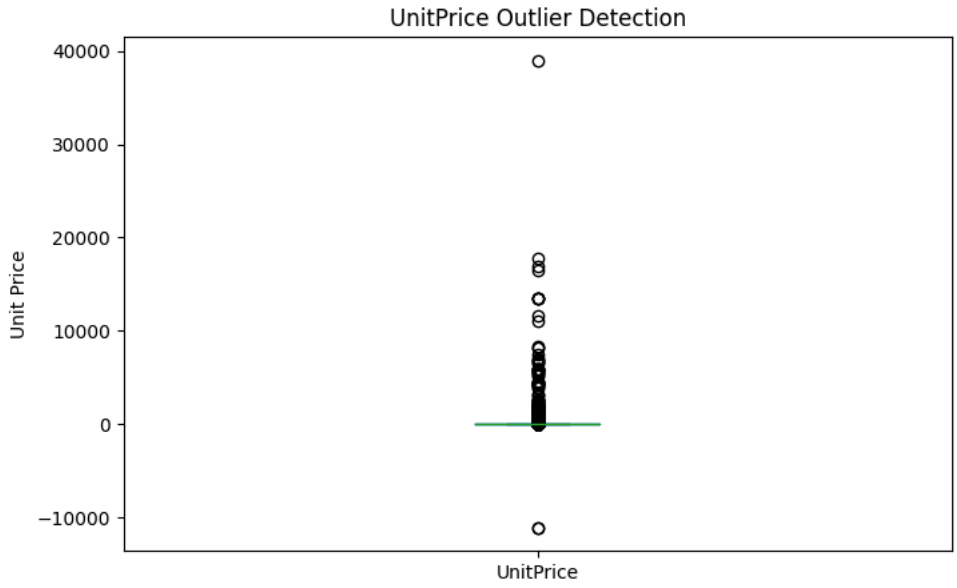
	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
16	536367	22622	Box Of Vintage Alphabet Blocks	2	2010-12-01 08:34:00	9.95	13047.0	United Kingdom
45	536370	POST	Postage	3	2010-12-01 08:45:00	18.00	12583.0	France
65	536374	21258	Victorian Sewing Box Large	32	2010-12-01 09:09:00	10.95	15100.0	United Kingdom
141	C536379	D	Discount	-1	2010-12-01 09:41:00	27.50	14527.0	United Kingdom
151	536382	22839	3 Tier Cake Tin Green And Cream	2	2010-12-01 09:45:00	14.95	16098.0	United Kingdom
152	536382	22838	3 Tier Cake Tin Red And Cream	2	2010-12-01 09:45:00	14.95	16098.0	United Kingdom
					2010-12-01			United

```
plt.figure(figsize=(8, 5))
```

```
df_clean["UnitPrice"].dropna().plot(kind="box")

plt.title("UnitPrice Outlier Detection")
plt.ylabel("Unit Price")

plt.show()
```




```
df_clean["Revenue"] = (
    df_clean["Quantity"] * df_clean["UnitPrice"]
)

df_clean["DiscountedPrice"] = (
    df_clean["UnitPrice"] * 0.90
)

df_clean["Year"] = df_clean["InvoiceDate"].dt.year
df_clean["Month"] = df_clean["InvoiceDate"].dt.month

print("New columns created successfully!")

display(
    df_clean[
        [
            "Quantity",
            "UnitPrice",
            "Revenue",
            "DiscountedPrice",
            "Year",
            "Month"
        ]
    ].head(10)
)
```

```
New columns created successfully!
```

	Quantity	UnitPrice	Revenue	DiscountedPrice	Year	Month
0	6	2.55	15.30	2.295	2010	12
1	6	3.39	20.34	3.051	2010	12
2	8	2.75	22.00	2.475	2010	12
3	6	3.39	20.34	3.051	2010	12
4	6	3.39	20.34	3.051	2010	12
5	2	7.65	15.30	6.885	2010	12
6	6	4.25	25.50	3.825	2010	12
7	6	1.85	11.10	1.665	2010	12
8	6	1.85	11.10	1.665	2010	12
9	32	1.69	54.08	1.521	2010	12

```
def categorize_price(price):

    if pd.isna(price):
        return "Unknown"

    elif price < 10:
        return "Low"

    elif price <= 100:
        return "Medium"

    else:
        return "High"


df_clean["PriceCategory"] = df_clean["UnitPrice"].apply(
    categorize_price
)

print("Price categories created!")

display(
    df_clean[
        ["UnitPrice", "PriceCategory"]
    ].head(15)
)

print("\nPrice category counts:")
print(df_clean["PriceCategory"].value_counts())
```

```
Price categories created!
```

	UnitPrice	PriceCategory
0	2.55	Low
1	3.39	Low
2	2.75	Low
3	3.39	Low
4	3.39	Low
5	7.65	Low
6	4.25	Low
7	1.85	Low
8	1.85	Low
9	1.69	Low
10	2.10	Low
11	2.10	Low
12	3.75	Low
13	1.65	Low
14	4.25	Low

```
Price category counts:
PriceCategory
Low       511668
Medium     23937
High        1036
Name: count, dtype: int64
```

```
# Check zero or negative quantities
print("Zero or negative quantities:")
display(df_clean[df_clean["Quantity"] <= 0].head(10))

# Check negative prices
print("\nNegative prices:")
display(df_clean[df_clean["UnitPrice"] < 0].head(10))

# Create a separate dataset for positive sales
df_sales = df_clean[
    (df_clean["Quantity"] > 0) &
    (df_clean["UnitPrice"] >= 0) &
    (df_clean["Quantity"].notna()) &
    (df_clean["UnitPrice"].notna())
].copy()

print("\nOriginal cleaned dataset rows:", len(df_clean))
print("Positive sales dataset rows:", len(df_sales))

display(df_sales.head())
```

Zero or negative quantities:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Revenue	DiscountedPrice	Year	Mo
141	C536379	D	Discount	-1	2010-12-01 09:41:00	27.50	14527.0	United Kingdom	-27.50	24.750	2010	
154	C536383	35004C	Set Of 3 Coloured Flying Ducks	-1	2010-12-01 09:49:00	4.65	15311.0	United Kingdom	-4.65	4.185	2010	
235	C536391	22556	Plasters In Tin Circus Parade	-12	2010-12-01 10:24:00	1.65	17548.0	United Kingdom	-19.80	1.485	2010	
236	C536391	21984	Pack Of 12 Pink Paisley Tissues	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom	-6.96	0.261	2010	
237	C536391	21983	Pack Of 12 Blue Paisley Tissues	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom	-6.96	0.261	2010	
238	C536391	21980	Pack Of 12 Red Retrospot Tissues	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom	-6.96	0.261	2010	
239	C536391	21484	Chick Grey Hot Water Bottle	-12	2010-12-01 10:24:00	3.45	17548.0	United Kingdom	-41.40	3.105	2010	
240	C536391	22557	Plasters In Tin Vintage Paisley	-12	2010-12-01 10:24:00	1.65	17548.0	United Kingdom	-19.80	1.485	2010	
241	C536391	22553	Plasters In Tin Skulls	-24	2010-12-01 10:24:00	1.65	17548.0	United Kingdom	-39.60	1.485	2010	
939	C536506	22960	Jam Making Set With Jars	-6	2010-12-01 12:38:00	4.25	17897.0	United Kingdom	-25.50	3.825	2010	

Negative prices:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Revenue	DiscountedPrice	Year
299983	A563186	B	Adjust Bad Debt	1	2011-08-12 14:51:00	-11062.06	NaN	United Kingdom	-11062.06	-9955.854	2011
299984	A563187	B	Adjust Bad Debt	1	2011-08-12 14:52:00	-11062.06	NaN	United Kingdom	-11062.06	-9955.854	2011

Original cleaned dataset rows: 536641
Positive sales dataset rows: 526052

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Revenue	DiscountedPrice	Year	Mont
0	536365	85123A	White Hanging Heart T-Light Holder	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom	15.30	2.295	2010	1

```
print("========== FINAL DATA VALIDATION ==========")

print("\nDataset shape:")
print(df_clean.shape)

print("\nMissing values:")
print(df_clean.isnull().sum())

print("\nDuplicate records:")
print(df_clean.duplicated().sum())

print("\nData types:")
print(df_clean.dtypes)

print("\nDescriptive statistics:")
display(df_clean.describe(include="all"))

new_columns = [
    "Revenue",
    "DiscountedPrice",
    "PriceCategory",
    "Year",
    "Month"
```

```
]

print("\nNew columns:")
display(df_clean[new_columns].head(10))
```

```
========== FINAL DATA VALIDATION ==========

Dataset shape:
(536641, 13)

Missing values:
InvoiceNo               0
StockCode               0
Description          1454
Quantity                0
InvoiceDate             0
UnitPrice               0
CustomerID         135037
Country                 0
Revenue                 0
DiscountedPrice         0
Year                    0
Month                   0
PriceCategory           0
dtype: int64

Duplicate records:
0

Data types:
InvoiceNo          string[python]
StockCode          string[python]
Description        string[python]
Quantity                    int64
InvoiceDate        datetime64[ns]
UnitPrice                 float64
CustomerID                float64
Country            string[python]
Revenue                   float64
DiscountedPrice           float64
Year                        int32
Month                       int32
PriceCategory              object
dtype: object

Descriptive statistics:
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Revenue
count	536641	536641	535187	536641.000000	536641	536641.000000	401604.000000	536641	536641.000000
unique	25900	4070	4194	NaN	NaN	NaN	NaN	38	NaN
top	573585	85123A	White Hanging Heart T-Light Holder	NaN	NaN	NaN	NaN	United Kingdom	NaN
freq	1114	2301	2357	NaN	NaN	NaN	NaN	490300	NaN
mean	NaN	NaN	NaN	9.620029	2011-07-04 08:57:06.087421952	4.632656	15281.160818	NaN	18.123861
min	NaN	NaN	NaN	-80995.000000	2010-12-01 08:26:00	-11062.060000	12346.000000	NaN	-168469.600000
25%	NaN	NaN	NaN	1.000000	2011-03-28 10:52:00	1.250000	13939.000000	NaN	3.750000
50%	NaN	NaN	NaN	3.000000	2011-07-19 14:04:00	2.080000	15145.000000	NaN	9.870000
75%	NaN	NaN	NaN	10.000000	2011-10-18 17:05:00	4.130000	16784.000000	NaN	17.400000
max	NaN	NaN	NaN	80995.000000	2011-12-09 12:50:00	38970.000000	18287.000000	NaN	168469.600000
std	NaN	NaN	NaN	219.130156	NaN	97.233118	1714.006089	NaN	380.656263

```
New columns:
```

	Revenue	DiscountedPrice	PriceCategory	Year	Month
0	15.30	2.295	Low	2010	12
1	20.34	3.051	Low	2010	12